SignSpeak: Real-Time American Sign Language Fingerspelling Recognition from a Webcam with a Convolutional Neural Network, Converted to Text and Speech

Ajay
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[email address]

A Navadeep Redddy
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[email address]

B Manideep
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[email address]

B Maheshwari
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[email address]

V. Pallavi
[DESIGNATION], Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[email address]

***Abstract*---**Sign language is the main medium of communication for many people with hearing and speech impairments, yet few hearing people understand it. This paper presents SignSpeak, a webcam-based system that recognises American Sign Language fingerspelling with a convolutional neural network, builds the letters into a sentence and speaks it aloud. Each frame is cropped to a hand region, segmented with Otsu thresholding, reduced to 64 x 64 pixels and classified by a 2.2-million-parameter network into 26 letters plus space, delete and nothing; a confidence threshold and a majority vote over recent frames decide when a letter is typed. Trained on 7,250 images of a public dataset on a desktop CPU, the network reaches 82.81% accuracy on a held-out split of 1,088 images, and the server pipeline needs about 53 ms per frame. Accuracy falls sharply when the hand is rotated by ten degrees and unseen signers were not tested, so real-world accuracy is expected to be lower. Words and motion signs are not covered.

***Index Terms*---**American Sign Language, assistive technology, convolutional neural network, fingerspelling, hand gesture recognition, image segmentation, real-time systems.

I. INTRODUCTION

Sign languages are the primary medium of communication for many deaf and hard-of-hearing people, but few hearing people can sign, and interpreters are scarce, costly and rarely present in spontaneous everyday exchanges [2], [3]. Automatic sign language recognition aims to close this gap by turning camera images into text or speech. Early systems used special sensors or restricted vocabularies [4], [5]; deep learning now classifies hand images directly [6], [7], and the most recent work targets video recognition of whole signs on small edge devices [1].

Today, a deaf person who needs to talk to a hearing person who does not sign relies on a human interpreter, on writing or typing, or on a phone application; each costs time or money, and none supports a quick exchange at a counter or in a classroom. Gloves and depth cameras read hand shape well but are expensive and rarely owned, whereas a plain webcam is built into almost every laptop. Fingerspelling, which spells a word letter by letter with the 26 hand shapes of the manual alphabet, is used for names and terms that have no sign of their own [6]; it is a bounded task with a small number of static classes, and therefore a sensible first target for a low-cost recogniser.

Two practical problems remain. Many published systems stop at a trained classifier evaluated on stored images; they are not packaged as a live application that a non-expert can run, and their accuracy is reported without the conditions that matter, such as background, lighting, hand rotation and signer. Second, recognition that is accurate on a benchmark often degrades with background clutter, changing light and unseen signers, which is the situation of every real use. This paper addresses the first problem and measures part of the second.

The contributions are: (1) a complete, tested system that runs a live webcam loop and turns recognised letters into an on-screen sentence and synthesised speech; (2) one shared preprocessing function (region of interest, Otsu segmentation, 64 x 64 normalisation) used for data collection, training and recognition; (3) a decision rule combining a confidence threshold, majority vote and stability count that turns noisy per-frame predictions into typed letters; and (4) a measured evaluation with accuracy, per-class results, a confidence-threshold analysis, robustness to rotation, brightness and noise, and latency, stated together with what was not tested. Section II reviews related work, Section III describes the method, Section IV reports results and Section V concludes.

A. System Hardware

All training and every measurement used one desktop computer: Intel Core i5-12500 (6 cores / 12 threads) (15.7 GB RAM), no graphics card, Windows 11. A standard webcam supplies the live video; the browser captures it.

B. Software Requirements

Python 3.11.9, TensorFlow/Keras 2.15.0 [8], OpenCV 4.11.0 [9], NumPy 1.26.4, a FastAPI service for the back

end, a React 18 and Vite interface for the browser, pyttsx3 for offline speech, and pytest for the automated tests.

II. LITERATURE SURVEY

Base paper. Vo et al. [1] study sign recognition on edge devices. Their input is not an image but a skeleton: a sequence of 27 upper-body and hand keypoints per frame, extracted by a pose estimator such as MediaPipe [10], processed by a spatio-temporal graph convolutional network [11]. Their method, MAG-KD, distils three large teacher networks into a small student (0.48 M parameters): a hard correctness mask removes teacher outputs that disagree with the ground truth, and a small gating network weights the remaining teachers, following the knowledge-distillation idea of Hinton et al. [12]. On the AUTSL Turkish Sign Language benchmark [13] (38,336 videos, 43 signers, 226 signs) the student reaches 94.25 +/- 0.19 % top-1 accuracy over three seeds, and 66.33 +/- 0.54 % on WLASL-100 [14]. Exported through ONNX and TensorRT in half precision it runs at 33.13 frames per second at 6.23 W on a Jetson Nano. The authors note two limits: the mask needs ground-truth labels, and the whole system is bounded by the speed and accuracy of the pose estimator.

Related work. Starner et al. [4] recognised sentence-level ASL from desk-mounted and wearable cameras with hidden Markov models; Pugeault and Bowden [5] classified fingerspelled letters in real time from colour and depth images of a Kinect sensor. Bheda and Radpour [6] applied deep convolutional networks to static images of letters and digits, building on the line of CNNs started by LeCun et al. [7]. Surveys [2], [3] separate static, isolated-word and continuous recognition and list background, lighting and signer variation as open problems.

From video to skeletons. Video methods began with 3D convolutional networks [15], which model motion directly but are expensive. Skeleton methods such as ST-GCN [11] pass only joint coordinates through a graph network, which is cheaper and less affected by background, but depends on pose estimates that jitter or drop joints in fast motion, the failure that the base paper's mask is designed to contain [1]. Both families take video sequences and are built for signs that move; a fingerspelled letter is nearly a static hand shape and can be classified from a single frame.

Datasets. AUTSL [13] and WLASL [14], the benchmarks of the base paper, contain word-level signs and do not apply to fingerspelling. For letters, the Kaggle ASL Alphabet dataset [16] offers 29 classes of 3,000 images each. Its images are frames from video, so neighbouring images are near-duplicates and no signer identifier is given; a split by image therefore says little about unseen signers, a caveat that shapes how the results below are reported.

Gap. The base paper and most recent work address word-level recognition from video, where a pose estimator and heavy training infrastructure are assumed. What is less often delivered is the simple end-to-end path for fingerspelling: an inexpensive camera, a small image classifier, and a live application that outputs text and speech, evaluated with its conditions stated. This work targets that path and does not attempt to match video methods.

TABLE I
REPRESENTATIVE WORK COMPARED

Work	Approach	Data	Reported result	Limitation
[1]	Skeleton GCN student, multi-teacher distillation	AUTSL, WLASL-100 (video)	94.25 % AUTSL; 66.33 % WLASL-100; 33.13 FPS	Needs pose estimator; words, not letters
[4]	Hidden Markov models on video	ASL sentences, small vocabulary	Early real-time ASL	Restricted vocabulary
[5]	Colour + depth, letter classifier	ASL letters, Kinect	Real-time letters	Depth sensor needed
[6]	Deep CNN on images	ASL letters, digits (static)	CNN works on static letters	No live application
[13]	Dataset and baselines	38,336 videos, 226 signs	Benchmark for [1]	Turkish signs
[3]	Survey of deep methods	Many datasets	Maps the field	No system

Reading Table I: the closest work in method is the base paper, and the closest in task is the line from Pugeault and Bowden to Bheda and Radpour, which classifies static fingerspelled letters. The reported results come from different datasets and metrics and are not ranked here; they show the range of what has been tried, not a leaderboard. The contrast that matters for this project is the input: skeleton sequences with distillation and half-precision deployment in [1], against a single segmented image and a small, unoptimised CNN here.

III. METHODOLOGY

Architecture. Fig. 1 shows the system. The browser owns the webcam: each frame is cover-cropped to 16:9, mirrored and drawn to a 960 x 540 canvas, compressed to JPEG (quality 0.8, about 27 KB) and posted to a FastAPI service, one request at a time and at least 60 ms apart. The service runs the same Python modules as the original desktop application: preprocessing, the network, the recogniser, speech and the training loop. Training images and the saved model are plain files; sessions are held in memory.

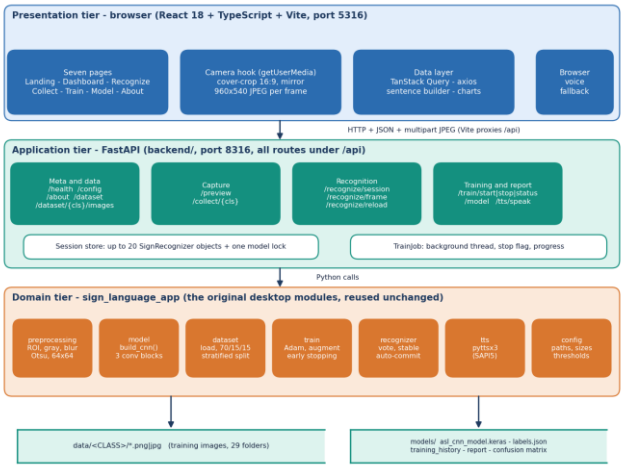


Fig. 1. System architecture: browser, FastAPI service, shared recognition modules and file storage.

Preprocessing. A 300 x 300 region of interest (x 620-920, y 90-390 of the 960 x 540 frame) is cropped, converted to grayscale and smoothed with a 7 x 7 Gaussian filter. Otsu's method [17] then picks the threshold t* that maximises the between-class variance of the grey-level histogram,

$$t^* = \arg\max\ \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2 \qquad (1)$$

where ω_0, ω_1 are the fractions and μ_0, μ_1 the mean grey levels of the two classes. If the mean of the resulting mask exceeds 127 it is inverted so that the hand is always white; one opening and two closings with a 3 x 3 kernel remove speckle and fill holes; the mask is resized to 64 x 64 and divided by 255. The same function serves data collection, training and recognition, so training and use cannot drift apart.

Data. The Kaggle ASL Alphabet dataset [16] has 87,000 training images (200 x 200 pixels, 29 classes of 3,000: letters A-Z, space, delete and nothing). Because neighbouring images are frames from video, an evenly spaced subset of 250 per class (7,250 images) was used, split 70/15/15 with a stratified random split (seed 42) into 5,075 training, 1,087 validation and 1,088 test images. No signer identifiers are provided, so the split is by image, not by person.

Model. The classifier (2,199,453 parameters, 26.48 MB) has three convolution blocks of 3 x 3 kernels with 32, 64 and 128 filters, each with batch normalisation [18], ReLU and 2 x 2 max pooling; dropout 0.25 follows blocks two and three. The 8,192-value feature vector feeds a dense layer of 256 units (batch normalisation, dropout 0.4) and a 29-way softmax,

$$p_k = \exp(z_k) / \Sigma_j \exp(z_j), \qquad (2)$$

$$L = -(1/N)\,\Sigma_i \Sigma_k\, y_{ik} \log p_{ik} \qquad (3)$$

trained with categorical cross-entropy and the Adam optimiser [19] (learning rate 10^{-3}, batch 32). Augmentation randomly rotates (12 degrees), shifts (10 %), zooms (12 %) and shears (8 %) the training images only; dropout [20] and early stopping on validation accuracy (patience 6) limit over-fitting. A small network was chosen because it trains on a CPU, at about 10 s per epoch, and classifies a frame in tens of milliseconds. The saved run used all 30 epochs.

Decision logic. A raw per-frame prediction is not a typed letter. As Fig. 2 shows, a prediction below 0.75 confidence is replaced by nothing; the last 15 accepted labels vote; and a letter is typed only after the winning label has been stable for 18 consecutive frames, differs from the last typed letter, and is not nothing. Showing nothing resets the repeat guard, so a double letter needs the hand to be lowered between letters. Space and delete are acted on as commands. The typed sentence can be edited and read aloud by an offline speech engine, with a browser voice as fallback.

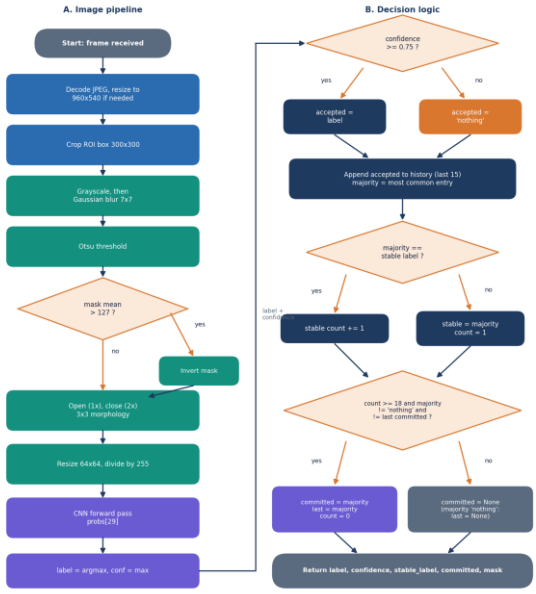


Fig. 2. Per-frame flow: image pipeline (left) and decision logic (right).

Service and implementation. The FastAPI service exposes 16 endpoints under one prefix: health, configuration and about; dataset listing and image access; a preview of the segmentation mask; capture of training images; recognition sessions and frames; model reload; speech; training start, stop and status; and the model report. Inputs are validated (epochs 1-500, batch 1-1,024, learning rate between 10^{-6} and 1, at least two classes with images, image-name patterns, speech text up to 2,000 characters) and errors return clear codes. A recognition session holds one recogniser object, up to 20 at a time, and a lock serialises calls to the single TensorFlow model; training runs in a background thread with a stop flag and progress that the browser polls. The React interface offers a live recognition page with the mask preview and sentence editor, a collection page that adds labelled images, a training page with live charts, and a model-performance page with per-class scores, the confusion matrix and training curves.

Evaluation protocol. Precision (P), recall (R) and F1 are computed per class on the test split, and the macro F1 is their mean over the 29 classes,

$$F1 = 2PR / (P + R), \quad \text{macro } F1 = (1/29)\,\Sigma_k F1_k. \qquad (4)$$

Top-3 accuracy counts a hit when the true class is among the three highest probabilities. For the confidence threshold, coverage is the fraction of frames whose top probability reaches the threshold and accuracy on accepted frames is measured on those frames only. Six evaluations are reported: the held-out split; 2,900 other archive photos (100 per class) that were never copied into the training folder; the 28 official Kaggle test photos; the threshold sweep; a robustness check on 580 photos (20 per class) under rotation, brightness, noise and blur; and per-stage latency over 300 frames.

Design rationale. Otsu's threshold needs no training and costs under a millisecond, but it assumes that hand and background fall into two grey-level classes; it was chosen for speed and simplicity, and its failure on cluttered scenes is

measured below. A 64 x 64 input keeps the first dense layer (8,192 x 256 weights, about 2.1 million of the parameters) small enough to train on a CPU while leaving finger shapes visible. The 18-frame stability rule equals about one second at the measured 17.6 frames per second, long enough to ignore the transition between two signs and short enough to type at a natural pace. Majority voting over 15 frames, rather than averaging probabilities, stops a single confident wrong frame from changing the output.

Reproducibility. Everything uses seed 42. One script measures every figure in Section IV from the saved model and writes it to a JSON file, and re-evaluating the saved model on the regenerated split reproduced the stored test accuracy (0.8281) and loss (0.5569). The collection page of the application adds labelled images from the webcam, so the system can be retrained on a new signer or background without code changes.

Verification. 52 pytest tests pass: 27 on the API (validation, errors, sessions, training control), 10 on preprocessing, 7 on the recogniser's voting and commit logic with a stub model, and 8 on dataset handling and the network. A 26-check live smoke test against a running server (input validation, collection of 80 images, a long run stopped at epoch two, a three-epoch training run, the model report, recognition and an unknown-session error) also passes, and every screen was walked through in a browser with an emulated camera fed real dataset photos.

IV. RESULTS

Test conditions. The headline figure is accuracy on the 1,088 test-split images, which were never used for training or model selection. Because the dataset consists of video frames from a few recordings, this is a split by image, not by signer, and it is not a test on unseen people. Table II lists the results. Accuracy is 82.81%, macro F1 0.830 and top-3 accuracy 92.9%; training reached 72.9% and validation 84.7% at epoch 30 (the cap), so more training would probably help.

With 1,088 test images the 95 % binomial interval around 82.8% is about +/-2.2 points (80.6% to 85.1%); this covers sampling noise only, not the choice of split or signer. Validation accuracy exceeds training accuracy because dropout and augmentation act only on the training pass, and the validation loss, which spiked in the first epochs while batch normalisation statistics settled, was stable from epoch six.

TABLE II
MEASURED ACCURACY BY PROTOCOL (ONE TRAINED MODEL)

Protocol	n	Accuracy	Macro F1	Top-3
Held-out test split (headline)	1,088	82.81%	0.830	92.9%
Other archive photos (optimistic)	2,900	87.24%	0.874	96.1%
Official Kaggle test photos	28	92.9% (26/28)	-	-
Robustness subset: clean	580	85.0%	-	-
Rotated +10 degrees	580	67.6%	-	-
Rotated -10 degrees	580	68.6%	-	-
Brightness x 1.5	580	73.3%	-	-
Gaussian noise (s = 25)	580	83.3%	-	-

The 2,900 other archive photos were not copied into the training folder, but they come from the same recordings as the training images and neighbouring frames are near-duplicates, so their 87.2% overstates generalisation and is not used as the headline. Of the 28 official Kaggle test photos, 26 are correct (A read as S at 0.675 confidence, I as D at 0.734).

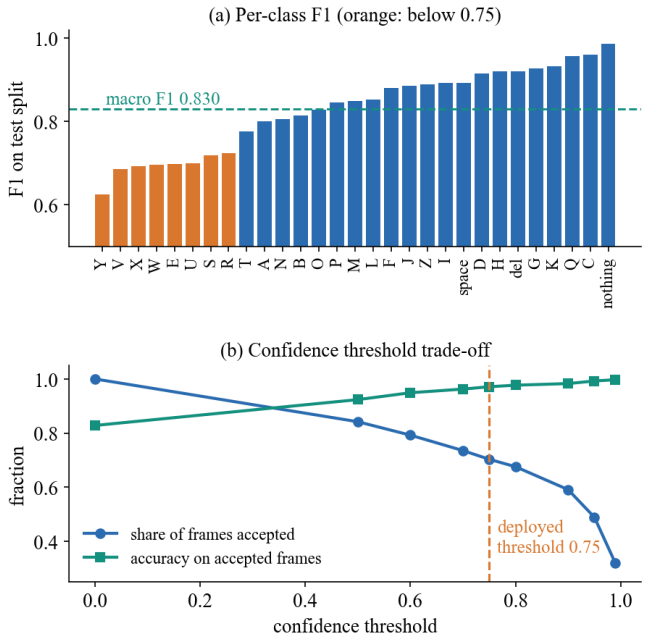


Fig. 3. Per-class F1 on the test split (a) and the effect of the confidence threshold (b).

Per-class results and failures. The easiest classes are nothing (F1 0.99), C and Q (0.96); the weakest are Y 0.62, V 0.69, X 0.69, W 0.70. The most frequent confusions are E read as B (7 images), V as W (7), V as U (6) and S as Y (6): signs that differ by finger position or are partly occluded. J and Z are motion letters but are read from one frame here. The segmentation masks still contain background structure in cluttered scenes, so part of what the network learns may be the scene rather than the hand.

Confident errors. A wrong letter can be shown with high confidence. In a sample of 1,160 archive photos, 147 were misread, and the most confident errors reached 97.5 % (B read as W) and 97.5 % (W read as Z). Confidence therefore reduces errors but does not remove them, which is why the stability rule and an editable sentence are part of the design.

Class difficulty. The weakest classes are the same under two protocols: on the 2,900 other photos the lowest per-class accuracies are E (64 %) and X (68 %), then U (79 %), which suggests that these are genuinely hard shapes for the model and not an artefact of one split. Top-3 accuracy of 92.9% against 82.8% at top-1 means the correct letter is often the runner-up, which an interface could exploit by offering the second-best letter as a one-touch correction.

Threshold. Requiring 0.75 confidence keeps 70.3% of test frames at 97.1% accuracy (all frames: 82.8%); 0.90 keeps 59.1% at 98.3%. Together with the 18-frame stability rule this suppresses most wrong letters at the cost of holding a sign for about a second.

Practical reading. With the deployed settings, about three frames in ten are discarded as uncertain and the rest are right about 97 times in 100, so a signer who holds a clear sign steady for about a second should see the correct letter typed; a hand held at an angle, a cluttered wall or a hard pair such as V and U or E and B will more often produce a wrong or missing letter, which the sentence editor lets the user correct.

Robustness and speed. Rotating the hand by +/-10 degrees lowers accuracy from 85.0% to about 68 %, and brighter light to 73.3%, while blur and moderate noise change little. Per frame (median of 300): JPEG decoding 1.67 ms, segmentation 0.83 ms, network 50.3 ms; server total 52.9 ms (95th percentile 57.9 ms) and a localhost HTTP round trip 56.7 ms, i.e. about 17.6 frames per second, with no model optimisation.

At about 17.6 frames per second a letter needs 18 stable frames, a little over one second, so the loop speed bounds typing to under about 60 letters per minute even when every frame is read correctly.

What the tests do not show. The browser path was exercised with an emulated camera fed real dataset photos, not a physical webcam, and no user study was run. Conditions outside the dataset's recordings, such as other cameras, lighting, hand distances and left-handed signers, were not evaluated. J and Z involve motion in ASL but are read from one frame here, so their scores (F1 0.89 and 0.89) describe still approximations of those signs.

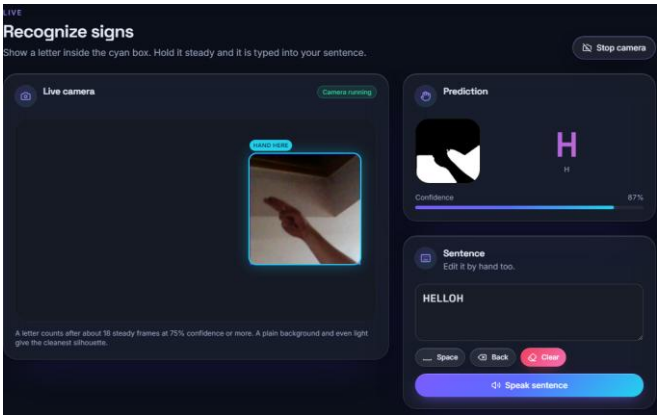


Fig. 4. Output screen: live recognition of the letter H at 87 % confidence, with the segmentation mask and the sentence being built.



Fig. 5. Output screen: model performance page with 82.8 % test accuracy and the 30-epoch training curves.

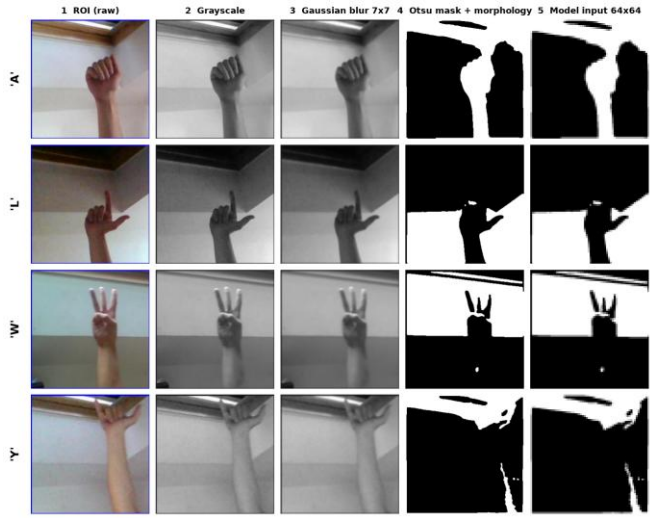


Fig. 6. Preprocessing stages for four letters: raw region of interest, grayscale, blur, Otsu mask with morphology, and the 64 x 64 model input. Background structure (ceiling edge, wall) survives in the masks.

Cost of training and what the masks show. Training on the CPU took about 9.9 s per epoch in a two-epoch benchmark (loading and preprocessing the 7,250 images took 75 s); the wall-clock time of the saved 30-epoch run was not recorded. The network call is 95 % of the server time, so faster inference needs a lighter or exported model, not faster image code. Fig. 6 shows why the model is sensitive to the scene: Otsu's global threshold separates the hand only where it contrasts with a plain background, and in a cluttered room the mask also contains walls and edges.

Threats to validity. Every number comes from one trained model and one split with one seed; repeated splits and seeds, which would show run-to-run variation, were not run. Only 250 of the 3,000 available images per class were used, hyper-parameters were not searched, and no ablation of the preprocessing choices (for example with and without segmentation) was made, so the contribution of each step is argued, not measured. The confidence interval above reflects sampling noise on the test split only.

Comparison with the base paper. The comparison is context only. The base paper [1] recognises word signs from

video skeletons (AUTSL: 226 signs, 94.25 %); SignSpeak recognises 29 static classes from single images (82.8%), with 2.2 M parameters (about 4.6 times the 0.48 M student) on a desktop CPU rather than a TensorRT-optimised Jetson Nano. Datasets, tasks, hardware and protocols differ, so no claim of superiority or inferiority is made.

V. CONCLUSION

SignSpeak recognises ASL fingerspelling letters from a webcam and turns them into text and speech. The network reaches 82.81% on a held-out split, and a 0.75 confidence threshold with a stability rule trades speed for reliability. Against the six project objectives: (1) the dataset covers the alphabet and three control classes but not common words; (2) preprocessing with segmentation, resizing and augmentation was done; (3) the CNN was designed and trained; (4) real-time operation was only partly met, at about 17.6 frames per second without model optimisation; (5) text and speech output work; and (6) accuracy, per-class results and latency were measured, but not on unseen signers. Limits: images come from a few recordings, so real-world accuracy will be lower; the model is sensitive to hand rotation; only letters are covered; and the live webcam path was verified with an emulated camera only.

Key findings: a 2.2-million-parameter CNN on thresholded hand masks learns the alphabet well enough for a first working system; the dominant weaknesses are hand rotation and cluttered backgrounds, not speed; and a confidence threshold with a stability rule raises the accuracy of accepted letters from about 83 % to about 97 %. The evidence supports the claim of a working prototype, not of everyday-use accuracy.

Intended use: SignSpeak is a communication aid and a research prototype, not a certified interpreter. A wrong letter can change a message, so the sentence is shown and can be edited before it is spoken.

Future work: test on unseen signers and varied backgrounds; train on all 3,000 images per class with stronger augmentation (rotation, brightness); use hand-landmark detection such as MediaPipe [10] instead of Otsu thresholding; add words and motion signs with a sequence model; and export the network to an optimised runtime for faster inference.

REFERENCES

[1] D. T. D. Vo, V. D. Nguyen, T. H. M. Nguyen and M. H. Vo, "MAG-KD: Mask-guided adaptive gating knowledge distillation for skeleton-based sign language recognition on edge devices," IEEE Access, vol. 14, pp. 54761-54775, 2026, doi: 10.1109/ACCESS.2026.3681176.

[2] B. A. Al Abdullah, G. A. Amoudi and H. S. Alghamdi, "Advancements in sign language recognition: A comprehensive review and future prospects," IEEE Access, vol. 12, pp. 128871-128895, 2024.

[3] R. Rastgoo, K. Kiani and S. Escalera, "Sign language recognition: A deep survey," Expert Syst. Appl., vol. 164, Art. no. 113794, Feb. 2021.

[4] T. Starner, J. Weaver and A. Pentland, "Real-time American Sign Language recognition using desk and wearable computer based video," IEEE Trans. Pattern Anal. Mach. Intell., vol. 20, no. 12, pp. 1371-1375, Dec. 1998.

[5] N. Pugeault and R. Bowden, "Spelling it out: Real-time ASL fingerspelling recognition," in Proc. IEEE Int. Conf. Comput. Vis. Workshops (ICCV Workshops), Barcelona, Spain, Nov. 2011, pp. 1114-1119.

[6] V. Bheda and D. Radpour, "Using deep convolutional networks for gesture recognition in American Sign Language," 2017, arXiv:1710.06836.

[7] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.

[8] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Oper. Syst. Design Implement. (OSDI), 2016, pp. 265-283.

[9] G. Bradski, "The OpenCV library," Dr. Dobb's J. Softw. Tools, 2000.

[10] C. Lugaresi et al., "MediaPipe: A framework for building perception pipelines," 2019, arXiv:1906.08172.

[11] S. Yan, Y. Xiong and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in Proc. AAAI Conf. Artif. Intell., vol. 32, no. 1, 2018.

[12] G. Hinton, O. Vinyals and J. Dean, "Distilling the knowledge in a neural network," 2015, arXiv:1503.02531.

[13] O. M. Sincan and H. Y. Keles, "AUTSL: A large scale multi-modal Turkish Sign Language dataset and baseline methods," IEEE Access, vol. 8, pp. 181340-181355, 2020.

[14] D. Li, C. R. Opazo, X. Yu and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison," in Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV), Mar. 2020, pp. 1448-1458.

[15] D. Tran, L. Bourdev, R. Fergus, L. Torresani and M. Paluri, "Learning spatiotemporal features with 3D convolutional networks," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), Dec. 2015, pp. 4489-4497.

[16] Akash, "ASL Alphabet," Kaggle dataset (grassknoted/asl-alphabet). [Online]. Available: https://www.kaggle.com/datasets/grassknoted/asl-alphabet (accessed Oct. 2026).

[17] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. SMC-9, no. 1, pp. 62-66, Jan. 1979.

[18] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd Int. Conf. Mach. Learn. (ICML), 2015, pp. 448-456.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015, arXiv:1412.6980.

[20] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," J. Mach. Learn. Res., vol. 15, no. 56, pp. 1929-1958, 2014.